

РУКОВОДСТВО ПО РАЗВЁРТЫВАНИЮ И СОПРОВОЖДЕНИЮ

Установка, настройка и эксплуатация

Аудитория: специалисты подразделения информационных технологий, системные
администраторы

Версия документа: 1.0

Дата: 28.07.2026

СОДЕРЖАНИЕ

1. Общие сведения.....	5
1.1. Назначение документа.....	5
1.2. Оформление.....	5
2. Термины.....	5
3. Требования к оборудованию и программному обеспечению.....	6
3.1. Конфигурация сервера.....	6
3.2. Дисковое пространство.....	7
3.3. Программное обеспечение сервера.....	7
3.4. Требования к рабочим местам.....	7
3.5. Сетевые требования.....	8
4. Состав поставки.....	8
4.1. Что передаётся.....	8
4.2. Структура каталога системы.....	9
4.3. Принцип наложения файлов описания.....	10
4.4. Чего в поставке нет.....	10
5. Установка.....	11
5.1. Подготовка сервера.....	11
5.2. Установка Docker.....	11
5.3. Размещение поставки.....	12
5.4. Создание файла параметров.....	13
5.5. Выбор имени базы данных.....	13
5.6. Подготовка сертификата.....	14
5.7. Первый запуск.....	14
5.8. Создание учётной записи администратора.....	15
5.9. Проверка после установки.....	15
6. Параметры конфигурации.....	16
6.1. Как задаются параметры.....	16
6.2. Обязательные параметры.....	17
6.3. Основные параметры.....	17
6.4. Параметры базы данных.....	18
6.5. Пересчёт параметров под другую конфигурацию.....	18

7. Защищённое соединение и сертификат	19
7.1. Зачем это нужно	19
7.2. Выбор способа получения сертификата	19
7.3. Выпуск сертификата	20
7.4. Установка корневого сертификата на рабочие места	21
7.5. Продление сертификата	22
8. Проверочный стенд	22
8.1. Назначение	22
8.2. Отличия от рабочего сервера	23
8.3. Развёртывание стенда	23
9. Эксплуатация рабочего сервера	24
9.1. Основные команды	24
9.2. Повседневные операции	24
9.3. Управление учётными записями	25
9.4. Восстановление доступа супер-администратора	25
9.5. Режим технических работ	25
9.6. Переименование базы данных	26
10. Порядок внесения изменений в систему	26
10.1. Общая схема	26
10.2. Получение исходного кода и внесение правки	27
10.3. Проверка изменения до выкладывания	27
10.4. Проверка на стенде	28
10.5. Выкладывание на рабочий сервер	28
10.6. Изменения схемы базы данных	29
10.7. Возврат к предыдущей версии	29
11. Резервное копирование и восстановление	30
11.1. Текущее состояние	30
11.2. Что подлежит копированию	30
11.3. Требования к процедуре	31
12. Мониторинг и журналы	31
12.1. Проверка состояния	31
12.2. Журналы приложения	32

12.3. Журналы внутри системы.....	33
12.4. Что контролировать.....	33
13. Диагностика и типовые неисправности.....	34
13.1. Порядок диагностики.....	34
13.2. Система недоступна, страница не открывается.....	34
13.3. Обратный прокси не запускается.....	34
13.4. Предупреждение о недоверенном сертификате.....	35
13.5. Прикладной сервер не запускается.....	35
13.6. Изменение параметра не подействовало.....	35
13.7. Заканчивается место на диске.....	36
13.8. Система работает медленно.....	36
13.9. Работник не видит раздел или получает отказ.....	36
13.10. Не приходят обновления без перезагрузки страницы.....	37
Приложение А. Порты и сетевые доступы.....	37
Приложение Б. Перечень параметров конфигурации.....	37
Приложение В. Файлы, необходимые для запуска.....	40
Приложение Г. Проверочный лист ввода в эксплуатацию.....	41

1. Общие сведения

1.1. Назначение документа

Документ содержит порядок развёртывания системы электронной подачи заявок «Бюро пропусков», её настройки, ввода в эксплуатацию и последующего сопровождения.

Документ рассчитан на специалиста, ранее с системой не работавшего. Порядок действий описан от состояния «есть сервер с установленной операционной системой» до состояния «система принимает заявки». Команды приведены в виде, пригодном для непосредственного выполнения, с указанием ожидаемого результата каждого шага.

Назначение системы, устройство и состав возможностей описаны в отдельном документе, «Техническое описание системы».

1.2. Оформление

Команды, имена файлов и параметры набраны моноширинным шрифтом. Угловые скобки означают подстановку: <домен> заменяется фактическим значением без скобок.

Сведения, требующие повышенного внимания, выделяются:

ПРИМЕЧАНИЕ. Пояснение, полезное для понимания, но не влияющее на порядок действий.

ВАЖНО. Пропуск приведёт к неверной работе или к дополнительной работе по исправлению.

ОПАСНО. Последствия необратимы. Утрата данных или утрата доступа к ним.

2. Термины

Таблица 2.1 — Термины, применяемые в документе

Термин	Значение
Рабочий сервер	Сервер, на котором система работает для сотрудников предприятия. В обиходе «прод», «продакшн»
Проверочный стенд	Отдельный сервер с той же системой для проверки изменений до выкладывания на рабочий. В обиходе «стенд», «стейдж»

Продолжение таблицы 2.1

Термин	Значение
Контейнер	Изолированная среда, в которой запускается одна часть системы. Части не мешают друг другу и не требуют установки в операционную систему
Образ	Заготовка, из которой запускается контейнер. Собирается из исходного кода один раз, затем запускается сколько угодно раз
Том	Хранилище данных контейнера, живущее отдельно от него. Пересоздание контейнера данные не затрагивает
Обратный прокси	Часть системы, принимающая соединения снаружи и распределяющая их между остальными частями
Сертификат	Электронный документ, которым сервер подтверждает браузеру свою подлинность и обеспечивает защищённое соединение
Удостоверяющий центр	Сторона, подписывающая сертификаты. Браузер доверяет сертификату, если доверяет подписавшему его центру
Файл параметров	Файл .env в каталоге системы, содержащий пароли, ключи и настройки конкретной установки

3. Требования к оборудованию и программному обеспечению

3.1. Конфигурация сервера

Конфигурация рассчитана на устойчивую работу при 1000-1500 одновременно работающих пользователей. Параметры базы данных и прикладного сервера, поставляемые в комплекте, подобраны именно под неё.

Таблица 3.1 — Рекомендуемая конфигурация сервера

Параметр	Значение
Процессор	6 физических ядер с частотой от 3,4 ГГц, например Intel Xeon E-2236
Оперативная память	32 ГБ с коррекцией ошибок
Дисковая подсистема	Два твердотельных накопителя от 480 ГБ в зеркальном массиве
Сеть	Один адрес, пропускная способность от 100 Мбит/с

ВАЖНО. Зеркальный массив обязателен. В базе хранятся персональные данные, и при отказе одиночного накопителя восстановление возможно только из резервной копии, то есть с потерей всех изменений, сделанных после её создания.

Минимальная конфигурация для пробной эксплуатации и проверочного стенда: 4 ядра, 8 ГБ памяти, 100 ГБ дискового пространства.

При такой конфигурации параметры базы данных из комплекта поставки требуется уменьшить, иначе службе не хватит памяти. Порядок изменения описан в подразделе 6.5.

3.2. Дисковое пространство

Таблица 3.2 — Ориентировочное потребление дискового пространства

Потребитель	Оценка
Образы контейнеров и операционная система	15-20 ГБ
База данных без журналов	1-3 ГБ за год работы
Журнал запросов	Основной потребитель. При высокой активности подробные записи за 30 суток занимают десятки гигабайт
Загруженные файлы	Фотографии, шаблоны бланков, приложения к заявкам
Резервные копии	Не менее трёх объёмов базы данных

Журнал запросов растёт быстрее остального, поэтому глубина хранения подробных записей вынесена в параметр `REQUEST_LOG_DETAIL_DAYS`. Уменьшение до 7-14 суток кратно снижает занимаемый объём и на работу системы не влияет.

3.3. Программное обеспечение сервера

Таблица 3.3 — Требования к программному обеспечению

Компонент	Требование
Операционная система	Любой дистрибутив Linux с поддержкой контейнеров: Ubuntu Server 22.04 и новее, Debian 12 и новее, RHEL-совместимые (Rocky, Alma), Astra Linux, РЕД ОС
Docker Engine	Версия 24 и новее
Docker Compose	Версия 2, вызывается как <code>docker compose</code>
openssl	Генерация секретов и сертификата
certbot	Только при выпуске сертификата через общедоступный удостоверяющий центр

Отдельно устанавливать Go, Node.js, PostgreSQL или nginx не требуется: всё необходимое собирается и работает внутри контейнеров.

3.4. Требования к рабочим местам

Браузер актуальной версии: Chrome, Yandex Browser, Firefox, Edge. Разрешение экрана от 1280 точек по ширине для рабочих мест бюро и постов. Интерфейс приспособлен и к мобильным устройствам.

Дополнительная установка чего-либо на рабочие места не требуется, за одним исключением: если сертификат сервера выпущен собственным удостоверяющим центром организации, потребуется однократно установить его корневой сертификат. Подробнее в подразделе 7.4.

3.5. Сетевые требования

Наружу открываются только два порта. Остальные порты частей системы существуют исключительно во внутренней сети контейнеров и не должны быть доступны с рабочих мест.

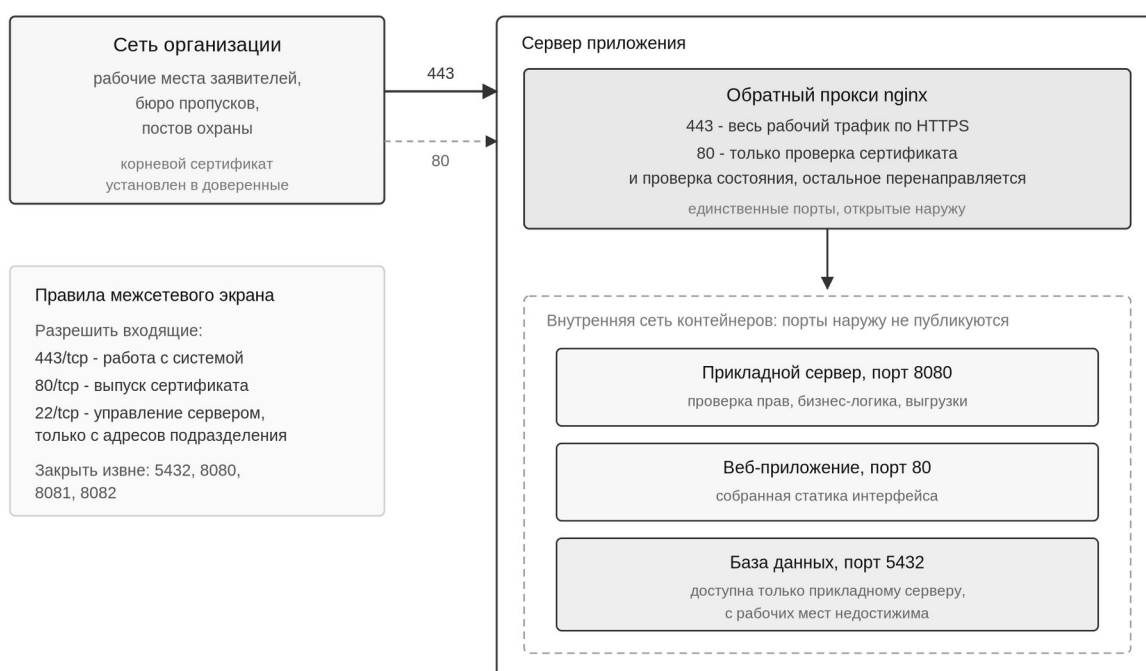


Рисунок 3.1 — Разграничение сетевых доступов

Полный перечень портов с указанием области доступности приведён в приложении А.

4. Состав поставки

4.1. Что передаётся

Поставка представляет собой архив с исходным кодом системы и всеми файлами, необходимыми для её сборки и запуска. Готовые собранные образы

не передаются: они собираются на сервере при установке, что исключает несовместимость с конкретной операционной системой.

Таблица 4.1 — Состав поставки

Элемент	Назначение
Исходный код серверной части и интерфейса	Из него на сервере собираются образы контейнеров
Файлы описания контейнеров	Определяют состав частей системы, их связи, ограничения ресурсов
Конфигурация обратного прокси	Правила распределения запросов, защищённое соединение, заголовки безопасности
Сборочный файл	Набор коротких команд управления вместо длинных команд Docker
Сценарий подготовки параметров	Создаёт файл параметров и генерирует пароли и ключи
Сценарий подготовки сертификата	Выпуск и продление сертификата, три способа на выбор
Эксплуатационная документация	Настоящий документ, техническое описание и руководства пользователей

Перечень конкретных файлов, без которых система не запустится, приведён в приложении В.

4.2. Структура каталога системы

После распаковки поставки в рабочем каталоге образуется следующая структура. Знать её требуется для изменения настроек и для диагностики.

Листинг 4.1 — Структура каталога системы

/opt/systemburo/	
├─ docker-compose.base.yml	состав служб, общая часть
├─ docker-compose.prod.yml	наложение: рабочий сервер
├─ docker-compose.staging.yml	наложение: стенд
├─ docker-compose.yml	локальная разработка
├─ Dockerfile	образ прикладного сервера
├─ Makefile	команды управления
├─ .env	параметры и секреты
├─ cmd/ internal/ docs/	код серверной части
├─ frontend/	
│ ├─ Dockerfile	образ веб-приложения
│ ├─ nginx.conf	отдача статики в образе
│ └─ src/ package.json	код интерфейса
├─ nginx/	
│ ├─ nginx.conf	прокси рабочего сервера
│ ├─ staging.conf	прокси стенда
│ ├─ Dockerfile.staging	образ прокси для стенда
│ ├─ certs/	сертификат и ключ
│ └─ acme-webroot/	выпуск сертификата
├─ scripts/	
│ ├─ init-env.sh	создание файла параметров
│ └─ init-tls.sh	выпуск и продление сертификата

4.3. Принцип наложения файлов описания

Ни рабочий сервер, ни стенд не имеют самостоятельного файла описания. Состав служб задан один раз в базовом файле, а различия среды добавляются наложением поверх него.

Листинг 4.2 — Порядок указания файлов описания

```
docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
ps
```

Именно поэтому во всех командах настоящего документа указываются два файла подряд. Порядок существенен: базовый идёт первым, наложение вторым, значения из второго заменяют значения из первого.

ВАЖНО. Команда, в которой указан только файл наложения, работать не будет. Служба базы данных определена в базовом файле, и без него Docker сообщит, что такой службы не существует. Это частая причина ошибок при копировании команд из посторонних источников.

Сокращённые команды из сборочного файла (make deploy-up, make deploy-logs и прочие) подставляют оба файла самостоятельно, поэтому при их использовании об этом думать не требуется.

4.4. Чего в поставке нет

В поставку намеренно не входят пароли, ключи шифрования и сертификаты. Они создаются на сервере при установке и остаются только у эксплуатирующей организации. Это означает, что разработчик не имеет доступа к данным работающей системы.

Не входят и данные: справочники, учётные записи, заявки. Система разворачивается пустой, после чего заполняется силами бюро пропусков через интерфейс.

5. Установка

Раздел описывает установку с нуля на сервер с установленной операционной системой. Результат: работающая система, доступная по защищённому соединению, с заведённой учётной записью администратора.

Команды выполняются от имени пользователя, входящего в группу `docker`.

5.1. Подготовка сервера

Установите служебные утилиты.

Листинг 5.1 — Установка служебных пакетов

```
# Ubuntu, Debian, Astra Linux
sudo apt update && sudo apt install -y openssl curl ca-certificates

# RHEL-совместимые, РЕД ОС
sudo dnf install -y openssl curl ca-certificates
```

Создайте отдельного пользователя для работы с системой. Работать под учётной записью суперпользователя не следует.

Листинг 5.2 — Создание пользователя

```
sudo useradd -m -s /bin/bash buro
sudo usermod -aG docker buro
```

Задайте часовой пояс: от него зависят отметки времени в журналах.

Листинг 5.3 — Установка часового пояса

```
sudo timedatectl set-timezone Europe/Moscow
```

5.2. Установка Docker

Версия из штатных репозиториев дистрибутива нередко устарела и может не поддерживать применяемый формат файлов описания. Устанавливайте из репозитория разработчика.

Листинг 5.4 — Установка Docker

```
# Ubuntu, Debian
curl -fsSL https://get.docker.com | sudo sh

# RHEL-совместимые
sudo dnf config-manager --add-repo \
  https://download.docker.com/linux/centos/docker-ce.repo
sudo dnf install -y docker-ce docker-ce-cli containerd.io \
  docker-compose-plugin
sudo systemctl enable --now docker
```

Для Astra Linux и РЕД ОС используйте пакеты из репозитория дистрибутива, если политика организации запрещает подключение внешних репозиторийев.

Проверьте установку.

Листинг 5.5 — Проверка установки Docker

```
docker --version
docker compose version
```

Ожидаемый результат: обе команды выводят номер версии. Вторая обязана вызываться именно как `docker compose`, через пробел. Если она сообщает об ошибке, а работает `docker-compose` через дефис, установлена устаревшая версия, и файлы описания системы обработаны не будут.

5.3. Размещение поставки

Листинг 5.6 — Распаковка поставки

```
sudo mkdir -p /opt/systemburo
sudo chown buro:buro /opt/systemburo
tar -xzf systemburo-<версия>.tar.gz -C /opt/systemburo --strip-components=1
cd /opt/systemburo
```

Все дальнейшие команды выполняются из этого каталога.

ВАЖНО. Имя каталога определяет имена томов с данными. Если каталог впоследствии переименовать, Docker посчитает установку новой и создаст пустые тома, а система запустится с пустой базой. Данные при этом не теряются, но чтобы полностью исключить такую ситуацию, добавьте в файл параметров строку `COMPOSE_PROJECT_NAME=systemburo` сразу после его создания. Тогда имена томов перестанут зависеть от имени каталога.

5.4. Создание файла параметров

Файл параметров создаётся сценарием, который сам генерирует стойкие случайные пароли и ключи. Задавать их вручную не следует.

Листинг 5.7 — Создание файла параметров

```
make init-production DOMAIN=<адрес системы>
```

Вместо <адрес системы> подставьте доменное имя, по которому система будет доступна, либо её сетевой адрес.

Ожидаемый результат: создан файл `.env`, на экран выведены сгенерированные учётные данные.

ОПАСНО. Сохраните выведенные значения в хранилище секретов организации до закрытия сессии. Повторно они нигде не отображаются, а восстановить их из файла паролей невозможно: там хранятся необратимые свёртки, а не сами пароли.

Особое внимание к параметру `DATA_ENCRYPTION_KEY`. Им зашифрованы сведения документов, удостоверяющих личность.

ОПАСНО. При утрате ключа шифрования расшифровать персональные данные невозможно. Резервная копия базы не поможет: она содержит те же зашифрованные значения. Ключ должен храниться отдельно от сервера и отдельно от резервных копий.

Сценарий намеренно отказывается перезаписывать существующий файл параметров.

ОПАСНО. Не запускайте создание параметров повторно на работающей системе. Будет сгенерирован новый ключ шифрования, и ранее зашифрованные персональные данные перестанут читаться.

5.5. Выбор имени базы данных

Имя базы задаётся параметром `DB_NAME` в файле параметров. Если требуется имя, отличное от заданного по умолчанию, измените его **до первого запуска**: тогда база сразу создастся с нужным именем.

Листинг 5.8 — Пример изменения имени базы до первого запуска

```
DB_NAME=buropropuskov  
DATABASE_URL=postgres://postgres:<пароль>@db/buropropuskov
```

Переименование уже работающей базы описано в подразделе 9.6.

5.6. Подготовка сертификата

Сертификат готовится до первого запуска: обратный прокси не стартует, если файла сертификата нет. Способы выпуска и выбор между ними описаны в разделе 7. Для установки внутри сети организации достаточно выполнить:

Листинг 5.9 — Выпуск сертификата для внутренней сети

```
./scripts/init-tls.sh self <адрес системы> <IP-адрес сервера>
```

Ожидаемый результат: на экран выведены сведения о выпущенном сертификате, включая срок действия, и указан путь к корневому сертификату для установки на рабочие места.

5.7. Первый запуск

Сборка образов выполняется на сервере и занимает от нескольких минут до получаса в зависимости от производительности.

Листинг 5.10 — Сборка и запуск

```
make deploy-build  
make deploy-up
```

Первый запуск дольше последующих: создаётся схема базы данных, устанавливаются расширения, строятся индексы.

Листинг 5.11 — Наблюдение за запуском

```
make deploy-logs
```

Ожидаемый результат: в журнале нет сообщений уровня `error`, последние строки сообщают о запуске сервера.

5.8. Создание учётной записи администратора

Листинг 5.12 — Создание администратора

```
make deploy-seed PASS=<пароль администратора>
```

Команда создаёт учётную запись `buorpropuskov` с правами супер-администратора и заполняет обязательные справочники: типы работников и разделы встроенного руководства.

ВАЖНО. Аргумент `PASS` обязателен. Без него будет установлен пароль по умолчанию, известный из состава поставки.

Пароль хранится в виде необратимой свёртки и в открытом виде нигде не сохраняется.

Демонстрационное наполнение (`make deploy-seed-demo`) на рабочем сервере запускать не следует: оно создаёт вымышленные заявки, организации и транспорт, которые придётся удалять вручную.

5.9. Проверка после установки

Выполните проверки последовательно. Каждая проверяет свой уровень, и успех предыдущей не гарантирует успех следующей.

Таблица 5.1 — Проверки после установки

Проверка	Команда или действие	Ожидаемый результат
Части системы запущены	<code>docker compose -f docker-compose.base.yml -f docker-compose.prod.yml ps</code>	Четыре службы в состоянии Up, база в состоянии healthy
Сервер отвечает	<code>curl -s http://localhost/health</code>	<code>{"status": "ok"}</code>
Защищённое соединение работает	<code>curl -sI https://<адрес>/</code>	Код 200, присутствует заголовок <code>Strict-Transport-Security</code>
Незащищённое перенаправляется	<code>curl -sI http://<адрес>/</code>	Код 301, заголовок <code>Location</code> начинается с <code>https://</code>
Вход выполняется	Открыть систему в браузере, войти под администратором	Открывается главный экран
Права применяются	Открыть раздел администрирования	Разделы управления доступны

ПРИМЕЧАНИЕ. Проверка `/health` подтверждает только то, что процесс жив. К базе данных она не обращается, поэтому успешный ответ при недоступной базе возможен. Полноценной проверкой является вход в систему.

6. Параметры конфигурации

6.1. Как задаются параметры

Параметры передаются прикладному серверу переменными окружения. Файл `.env` в каталоге системы подставляет значения в описание контейнеров.

ВАЖНО. Описание рабочего сервера передаёт в контейнер не весь файл параметров, а только явно перечисленный набор. Параметры вне этого набора в контейнер не попадают и работают на встроенных значениях, даже если прописаны в `.env`.

Через файл параметров действуют: `DATABASE_URL`, `JWT_SECRET`, `JWT_REFRESH_SECRET`, `JWT_ACCESS_TTL`, `JWT_REFRESH_TTL`, `LOG_LEVEL`, `DATA_ENCRYPTION_KEY`, `CORS_ALLOWED_ORIGINS`, `SWAGGER_ENABLED`, а также `DB_USER`, `DB_PASSWORD`, `DB_NAME`, из которых собирается строка подключения.

Чтобы изменить любой другой параметр, добавьте его в раздел `environment` службы `backend` файла `docker-compose.prod.yml` и перезапустите систему.

Листинг 6.1 — Изменение параметра, не передаваемого через файл параметров

```
backend:
  environment:
    RESET_TIMEZONE: "Asia/Yekaterinburg"
    REQUEST_LOG_DETAIL_DAYS: "14"
```

Листинг 6.2 — Применение изменений

```
make deploy-up
```

Ожидаемый результат: контейнер прикладного сервера пересоздан, новые значения действуют. Проверить можно командой из подраздела 13.6.

6.2. Обязательные параметры

Три параметра обязательны. При их отсутствии или недопустимом значении прикладной сервер завершает работу с ошибкой, а не запускается в ослабленном режиме.

Таблица 6.1 — Обязательные параметры

Параметр	Проверка при запуске
DATABASE_URL	Должен начинаться с postgres:// или postgresql://
JWT_SECRET	Не короче 32 символов
JWT_REFRESH_SECRET	Не короче 32 символов, отличается от предыдущего

Раздельные ключи для маркера доступа и маркера продления принципиальны: при совпадении маркер продления можно было бы предъявить вместо маркера доступа и продлить сеанс бесконечно.

6.3. Основные параметры

Полный перечень приведён в приложении Б. Здесь перечислены параметры, которые чаще всего требуется изменять.

Таблица 6.2 — Часто изменяемые параметры

Параметр	По умолчанию	Назначение
LOG_LEVEL	info	Подробность журнала: debug, info, warn, error
RESET_TIMEZONE	Europe/Moscow	Часовой пояс суточного сброса территориальных статусов
REQUEST_LOG_DETAIL_DAYS	30	Глубина хранения подробных записей журнала запросов
RATE_LIMIT_PER_MINUTE	200	Допустимое число запросов в минуту от одного работника
UPLOAD_MAX_FILE_SIZE	10485760	Наибольший размер загружаемого файла, 10 МБ
SWAGGER_ENABLED	false	Публикация описания программного интерфейса
REQUIRE_ENCRYPTION	false	Запрет запуска без ключа шифрования персональных данных

ВАЖНО. На рабочем сервере установите REQUIRE_ENCRYPTION в значение true. Это исключает ситуацию, когда система запустилась без ключа шифрования и записала персональные данные в открытом виде.

6.4. Параметры базы данных

Заданы в файле описания рабочего сервера и подобраны под конфигурацию из подраздела 3.1.

Таблица 6.3 — Параметры базы данных и среды выполнения

Параметр	Значение	Смысл
shared_buffers	8GB	Память под собственный кэш страниц базы
effective_cache_size	20GB	Оценка доступной памяти под кэш, влияет на выбор плана запроса
work_mem	32MB	Память на одну операцию сортировки или соединения
maintenance_work_mem	1GB	Память под построение индексов и обслуживание
max_connections	150	Наибольшее число одновременных подключений
max_wal_size	4GB	Объём журнала предзаписи между контрольными точками
random_page_cost	1.1	Стоимость произвольного чтения, значение для твердотельных накопителей
effective_io_concurrency	200	Число параллельных операций ввода-вывода
GOMAXPROCS	6	Число ядер, используемых прикладным сервером
GOMEMLIMIT	6GiB	Порог, при котором усиливается освобождение памяти

6.5. Пересчёт параметров под другую конфигурацию

При установке на сервер с иными характеристиками параметры базы данных требуется пересчитать, иначе службе не хватит памяти либо она не использует доступную.

Таблица 6.4 — Правила пересчёта

Параметр	Правило
shared_buffers	Около четверти оперативной памяти
effective_cache_size	Около двух третей оперативной памяти
maintenance_work_mem	Около 1/32 оперативной памяти
GOMAXPROCS	Равно числу ядер процессора
GOMEMLIMIT	Около одной пятой оперативной памяти

Пример для сервера с 8 ГБ памяти и 4 ядрами: `shared_buffers=2GB`, `effective_cache_size=5GB`, `maintenance_work_mem=256MB`, `GOMAXPROCS=4`, `GOMEMLIMIT=1600MiB`.

7. Защищённое соединение и сертификат

7.1. Зачем это нужно

Система работает только по защищённому соединению. Причина не формальная: через неё передаются пароли и персональные данные, которые по незащищённому соединению видны любому, кто имеет доступ к сети между рабочим местом и сервером.

Кроме того, продление сеанса технически работает только по защищённому соединению: браузеру запрещено сохранять маркер продления, полученный по обычному протоколу. Поэтому без сертификата вход в систему не заработает вовсе.

ПРИМЕЧАНИЕ. Сертификат нужен серверу, а не пользователям. Сотрудникам выдаются обычные имя и пароль. Никаких персональных сертификатов сотням работников выдавать не требуется.

7.2. Выбор способа получения сертификата

Сертификат должен быть подписан стороной, которой доверяет браузер. Отсюда три возможных способа, различающихся тем, кто подписывает.

Таблица 7.1 — Способы получения сертификата

Способ	Когда подходит	Что потребуется на рабочих местах
Собственный удостоверяющий центр	Система работает внутри сети, внешнего доменного имени нет	Однократная установка одного корневого сертификата, централизованно групповой политикой
Общедоступный удостоверяющий центр	У организации есть доменное имя, которым она владеет	Ничего, доверие настроено в браузерах изначально
Удостоверяющий центр организации	В организации развёрнута собственная служба сертификатов	Ничего, корневой сертификат уже установлен на машинах домена

ПРИМЕЧАНИЕ. Второй способ часто оказывается удобнее первого, и он применим, даже когда сервер стоит внутри сети и недоступен извне. Владение доменом подтверждается записью в системе доменных имён, а не доступом к серверу. Если у организации есть свой домен, этот путь избавляет от установки чего-либо на рабочие места.

7.3. Выпуск сертификата

Собственный удостоверяющий центр. Применяется при работе внутри сети организации.

Листинг 7.1 — Выпуск сертификата собственным центром

```
./scripts/init-tls.sh self <адрес системы> <IP-адрес сервера>
```

Сценарий создаёт корневой сертификат удостоверяющего центра и подписанный им сертификат сервера. Указание сетевого адреса вторым аргументом позволяет обращаться к системе и по имени, и по адресу.

Ожидаемый результат: выведены сведения о сертификате и путь к корневому сертификату `nginx/certs/ca/ca.crt`.

Общедоступный удостоверяющий центр. Применяется при наличии доменного имени, доступного из сети общего пользования, и открытого порта 80.

Листинг 7.2 — Выпуск сертификата общедоступным центром

```
./scripts/init-tls.sh self <домен>  
make deploy-up  
./scripts/init-tls.sh acme <домен> <адрес электронной почты>
```

Порядок двухшаговый: сначала создаётся временный сертификат, чтобы прокси смог запуститься, затем выпускается настоящий. Без первого шага прокси не стартует, а без работающего прокси невозможно подтвердить владение доменом.

Готовый сертификат. Применяется, если сертификат приобретён либо выпущен удостоверяющим центром организации.

```
./scripts/init-tls.sh import <файл цепочки> <файл ключа>
```

Сценарий сверяет соответствие ключа сертификату и отказывается устанавливать несовпадающую пару. Это защищает от типичной ошибки, при которой прокси перестаёт запускаться уже после перезапуска, когда разбираться некогда.

7.4. Установка корневого сертификата на рабочие места

Шаг обязателен **только** при выпуске сертификата собственным удостоверяющим центром.

ВАЖНО. Без этого шага работники увидят предупреждение о недоверенном сертификате и не смогут его пропустить. Кнопки «перейти всё равно» не будет: система требует обязательного защищённого соединения, а это запрещает браузеру такой обход. Установка корневого сертификата снимает предупреждение полностью.

Файл для установки: `nginx/certs/ca/ca.crt`. Он одинаков для всех рабочих мест.

Централизованно в домене. Групповая политика: «Конфигурация компьютера» → «Политики» → «Конфигурация Windows» → «Параметры безопасности» → «Политики открытого ключа» → «Доверенные корневые центры сертификации», импорт файла. Политика применится на рабочих местах при очередном обновлении.

Отдельная машина Windows. Из командной строки с правами администратора:

```
certutil -addstore -f Root ca.crt
```

Рабочее место Linux.

```
sudo cp ca.crt /usr/local/share/ca-certificates/buro-ca.crt  
sudo update-ca-certificates
```

ОПАСНО. Приватный ключ удостоверяющего центра `nginx/certs/ca/ca.key` позволяет выпустить сертификат на любое имя, которому будут доверять все рабочие места организации. Он не должен покидать сервер и попадать в резервные копии общего доступа.

7.5. Продление сертификата

Сертификат, выпущенный собственным центром, действует 825 суток. За месяц до истечения выпустите новый той же командой: корневой сертификат переиспользуется, поэтому повторно устанавливать его на рабочие места не потребуется.

Листинг 7.6 — Продление сертификата собственного центра

```
./scripts/init-tls.sh self <адрес системы> <IP-адрес сервера>  
docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \  
exec nginx nginx -s reload
```

Сертификат общедоступного центра действует 90 суток и требует регулярного продления. Настройте задание в планировщике.

Листинг 7.7 — Задание на автоматическое продление

```
30 3 * * * cd /opt/systemburo && ./scripts/init-tls.sh renew \  
>> /var/log/systemburo-tls.log 2>&1
```

Команда безопасна для ежедневного запуска: она обращается за новым сертификатом только когда до истечения остаётся менее 30 суток, и перезагружает прокси лишь при фактическом обновлении файла.

Листинг 7.8 — Проверка срока действия

```
openssl x509 -in nginx/certs/fullchain.pem -noout -dates
```

Ожидаемый результат: две строки с датами начала и окончания действия.

8. Проверочный стенд

8.1. Назначение

Проверочный стенд представляет собой второй экземпляр системы, развёрнутый отдельно от рабочего сервера. Он нужен, чтобы проверять

изменения и настройки до их применения на рабочем сервере, обучать сотрудников без риска для данных и воспроизводить обращения пользователей.

ВАЖНО. Не разворачивайте стенд на том же сервере, где работает рабочий экземпляр. Они разделят память и дисковую подсистему, и проверка нагрузки на стенде скажется на работе сотрудников.

8.2. Отличия от рабочего сервера

Таблица 8.1 — Отличия стенда от рабочего сервера

Свойство	Стенд	Рабочий сервер
Доступ	Закрит запросом имени и пароля на входе	Открыт, разграничение внутри системы
Подробность журнала	debug	info
Панель управления базой данных	Разворачивается, доступна по адресу /pgadmin/	Не разворачивается
Наполнение	Допустимы демонстрационные данные	Только рабочие данные

ПРИМЕЧАНИЕ. Запрос имени и пароля на входе в стенд решает одну задачу, не пустить посторонних. Он не заменяет разграничение прав внутри системы.

8.3. Развёртывание стенда

Порядок совпадает с рабочим сервером, отличаются вызываемые команды.

Листинг 8.1 — Развёртывание стенда

```
make init-staging DOMAIN=<адрес стенда>
./scripts/init-tls.sh self <адрес стенда>
make staging-build
make staging-up
make staging-seed PASS=<пароль администратора>
```

Ожидаемый результат: стенд доступен по указанному адресу, при открытии запрашивает имя и пароль, выведенные сценарием подготовки параметров.

Демонстрационное наполнение добавляется отдельно:

```
make staging-seed-demo PASS=<пароль администратора>
```

9. Эксплуатация рабочего сервера

9.1. Основные команды

Таблица 9.1 — Команды управления системой

Действие	Команда
Запуск	make deploy-up
Остановка	make deploy-down
Пересборка образов	make deploy-build
Просмотр журналов	make deploy-logs
Состояние служб	docker compose -f docker-compose.base.yml -f docker-compose.prod.yml ps

Остановка прерывает работу пользователей. Прикладной сервер завершается корректно: перестаёт принимать новые запросы, даёт до 10 секунд на завершение текущих и только затем останавливается, поэтому незавершённых операций записи при штатной остановке не остаётся.

9.2. Повседневные операции

Таблица 9.2 — Регламент повседневных работ

Операция	Периодичность	Кто выполняет
Контроль состояния служб	Ежедневно	Подразделение ИТ
Просмотр журнала на ошибки	Ежедневно	Подразделение ИТ
Просмотр журнала отказов в доступе	Еженедельно	Подразделение ИТ либо оператор системы с правом просмотра
Контроль свободного места	Еженедельно	Подразделение ИТ
Контроль срока действия сертификата	Ежемесячно	Подразделение ИТ
Проверка восстановления из резервной копии	Ежемесячно	Подразделение ИТ

ПРИМЕЧАНИЕ. Наблюдение за журналами системы и разбор обращений пользователей доступны не только подразделению ИТ, но и операторам бюро пропусков, если им выдано соответствующее право. Журналы открываются прямо в интерфейсе системы, доступ к серверу для этого не нужен.

9.3. Управление учётными записями

Заведение и блокировка работников выполняются в интерфейсе системы и описаны в руководстве администратора. С точки зрения эксплуатации существенны два обстоятельства.

Блокировка действует немедленно: заблокированный работник теряет все права, а его сеансы прекращаются, поэтому продолжить работу в уже открытой вкладке он не сможет.

Изменение прав вступает в силу с задержкой до 30 секунд, поскольку результат вычисления прав кэшируется. Для открытых страниц с автообновлением задержка может достигать 10 минут.

9.4. Восстановление доступа супер-администратора

Супер-администратор в системе один. При утрате доступа к нему восстановление выполняется повторным запуском команды создания учётной записи: она не создаёт вторую запись, а обновляет пароль существующей.

Листинг 9.1 — Смена пароля супер-администратора

```
make deploy-seed PASS=<новый пароль>
```

Ожидаемый результат: команда завершается без ошибок, вход выполняется с новым паролем. Прочие данные не затрагиваются.

9.5. Режим технических работ

Режим включается супер-администратором в разделе системного управления. При включённом режиме все работники, кроме супер-администратора, получают уведомление о недоступности системы и не могут выполнять действия.

Режим следует включать перед обновлением и любыми работами, затрагивающими целостность данных. Он не останавливает фоновые задачи и не прерывает уже выполняющиеся запросы.

9.6. Переименование базы данных

Если имя базы требуется изменить уже после запуска системы, порядок такой.

Листинг 9.2 — Переименование базы данных

```
make deploy-down
docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  up -d db
docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  exec db psql -U postgres \
  -c 'ALTER DATABASE auto_registry RENAME TO buropropuskov;'
```

Затем измените в файле параметров DB_NAME и имя базы в конце строки DATABASE_URL, после чего запустите систему.

Листинг 9.3 — Запуск после переименования

```
make deploy-up
```

Ожидаемый результат: система запускается, данные на месте. Операция мгновенная, данные не копируются.

ВАЖНО. Переименование выполняется при остановленном прикладном сервере. При активных подключениях база откажется переименовываться.

10. Порядок внесения изменений в систему

Раздел описывает полный путь изменения: от получения исходного кода до его появления на рабочем сервере.

10.1. Общая схема

Таблица 10.1 — Этапы внесения изменения

Этап	Где выполняется	Результат
1. Получение исходного кода	Рабочее место разработчика	Локальная копия проекта
2. Внесение правки	Рабочее место разработчика	Изменённый код в отдельной ветке
3. Автоматическая проверка	Конвейер сборки	Отчёт о прохождении проверок
4. Проверка на стенде	Проверочный стенд	Подтверждение работы вживую
5. Выкладывание на рабочий сервер	Рабочий сервер	Изменение доступно пользователям

10.2. Получение исходного кода и внесение правки

Исходный код хранится в системе контроля версий Git. Работа ведётся в отдельной ветке, а не в основной: это позволяет проверить изменение до того, как оно попадёт к другим.

Листинг 10.1 — Получение кода и создание ветки

```
git clone <адрес репозитория> systemburo
cd systemburo
git checkout -b fix/kratkoe-opisanie
```

Локальный запуск для разработки поднимает систему с отдельной базой и не затрагивает ни стенд, ни рабочий сервер.

Листинг 10.2 — Локальный запуск для разработки

```
make up
make seed
```

Ожидаемый результат: интерфейс доступен на порту 8081, программный интерфейс на 8080, вход под учётной записью, созданной командой заполнения.

10.3. Проверка изменения до выкладывания

Перед отправкой изменения прогоняются тесты. Локально достаточно прогнать те, что относятся к изменённой части.

Листинг 10.3 — Прогон проверок

```
make test                # бэкенд-тесты
cd frontend && npm run test:run  # фронтенд-тесты
cd frontend && npx playwright test # E2E-сценарии
```

Ожидаемый результат: все тесты завершаются успешно. Упавший тест выводит своё имя и расхождение ожидаемого с фактическим.

После отправки изменения конвейер запускает полный набор тестов автоматически, включая сборку образов и сканирование уязвимостей. Изменение, не прошедшее тесты, к выкладыванию не допускается.

10.4. Проверка на стенде

После прохождения тестов и включения изменения в основную ветку оно автоматически выкладывается на проверочный стенд.

На стенде проверяются сценарии, которые автоматика не покрывает: внешний вид, удобство, поведение на реальных данных.

Таблица 10.2 — Проверяемые на стенде сценарии

Сценарий	Что подтверждает
Вход администратора и рядового работника	Проверку подлинности и выдачу прав
Подача заявки со всеми типами вложений	Основной рабочий процесс
Приём в работу, согласование, завершение	Переходы состояний и запись решений
Открытие таблицы поста, отметка въезда и выезда	Работу постов
Формирование печатной формы по заявке	Работу шаблонов бланков
Выгрузка отчёта	Формирование электронных таблиц
Открытие панели показателей	Расчёт аналитики

10.5. Выкладывание на рабочий сервер

ОПАСНО. Перед выкладыванием создайте резервную копию базы данных. Это единственный способ вернуться назад гарантированно, см. подраздел 10.7.

Порядок действий на рабочем сервере.

1. Предупредить пользователей, при длительном обновлении включить режим технических работ (подраздел 9.5).
2. Получить новую версию исходного кода в каталог системы.

Листинг 10.4 — Обновление исходного кода на рабочем сервере

```
cd /opt/systemburo
git fetch origin
git checkout <версия>
```

1. Пересобрать образы и перезапустить систему.

Листинг 10.5 — Пересборка и перезапуск

```
make deploy-build
make deploy-up
```

1. Проследить за журналом запуска.

```
make deploy-logs
```

Ожидаемый результат: в журнале нет сообщений уровня `error`. Схема базы приводится в соответствие автоматически именно на этом шаге, и здесь же проявятся несовместимости, если они есть.

1. Выполнить проверки из подраздела 5.9.
2. Снять режим технических работ.

Пользователи не смогут работать с момента перезапуска до успешного старта, обычно это одна-две минуты.

10.6. Изменения схемы базы данных

Схема приводится в соответствие автоматически при каждом запуске прикладного сервера, отдельной команды не требуется. Практические следствия:

- изменения применяются в момент запуска, и если они объёмны, запуск затянется;
- изменения носят добавляющий характер: создаются таблицы, столбцы и индексы, удаления не выполняются;
- при ошибке приведения схемы прикладной сервер завершает работу и не запускается в частично обновлённом состоянии.

10.7. Возврат к предыдущей версии

Возврат кода выполняется получением прежней версии и повторной сборкой.

```
cd /opt/systemburo  
git checkout <предыдущая версия>  
make deploy-build  
make deploy-up
```

ВАЖНО. Приведение схемы базы выполняется только вперёд. Изменения схемы, внесённые новой версией, при откате кода не отменяются. Поскольку они носят добавляющий характер, прежняя версия обычно продолжает работать. Но если обновление изменило смысл существующих данных, потребуется восстановление базы из копии, созданной до обновления.

11. Резервное копирование и восстановление

11.1. Текущее состояние

Штатная процедура резервного копирования в составе системы отсутствует. Резервное копирование обеспечивается силами эксплуатирующей организации.

ОПАСНО. До настройки резервного копирования отказ дисковой подсистемы или ошибочная операция приведут к безвозвратной утрате всех данных, включая персональные.

Резервное копирование, предоставляемое как услуга хостинга, работает на уровне образа сервера целиком. Оно позволяет вернуть сервер в прежнее состояние, но не заменяет копию базы данных: из образа нельзя извлечь отдельную таблицу, отменить последствия ошибочной операции или перенести данные на другой сервер.

11.2. Что подлежит копированию

Копирования одной базы данных недостаточно.

Таблица 11.1 — Состав данных для резервного копирования

Элемент	Где расположен	Последствия утраты
База данных	Том pgdata	Полная утрата заявок, реестров, журналов, учётных записей
Загруженные файлы	Том uploads	Утрата фотографий, шаблонов бланков, приложений к заявкам. Записи в базе останутся со ссылками на отсутствующие файлы
Файл параметров	.env в каталоге системы	Утрата ключа шифрования персональных данных
Сертификат удостоверяющего центра	nginx/certs/	Потребуется повторный выпуск и повторная установка корневого сертификата на все рабочие места

11.3. Требования к процедуре

- Копия базы создаётся логической выгрузкой средствами СУБД, а не копированием файлов работающей базы: такая копия непригодна для восстановления.
- Ключ шифрования персональных данных хранится отдельно от копий базы. При хранении вместе компрометация архива означает компрометацию персональных данных.
- Копии, содержащие персональные данные, защищаются наравне с самой системой.
- Копии хранятся вне сервера системы: копия на том же диске не защищает от отказа диска.
- Восстановимость проверяется регулярно. Копия, из которой ни разу не восстанавливались, резервной копией не является.

12. Мониторинг и журналы

12.1. Проверка состояния

Адрес `/health` возвращает признак работоспособности прикладного сервера. Проверка подключается к средствам мониторинга организации.

Листинг 12.1 — Проверка состояния

```
curl -s http://localhost/health
```

Ожидаемый результат: `{"status": "ok"}`.

ВАЖНО. Проверка подтверждает, что процесс запущен и отвечает, но к базе данных она не обращается. При недоступной базе проверка продолжит отвечать успешно, а система работать не будет. Полноценным контролем является периодическая проверка входа в систему.

Контейнер прикладного сервера имеет встроенную проверку работоспособности с интервалом 30 секунд. Контейнер базы данных проверяется каждые 10 секунд, и прикладной сервер не запускается, пока база не сообщит о готовности.

12.2. Журналы приложения

Прикладной сервер пишет журнал в поток вывода и, если задан параметр `LOG_FILE_PATH`, дополнительно в файл. На рабочем сервере файловый журнал включён.

Смена файлов выполняется самим приложением, настраивать системное средство ротации не требуется: файл сменяется при достижении 100 МБ, хранится 30 суток, сохраняется до 14 файлов, старые сжимаются.

Листинг 12.2 — Просмотр журналов

```
# все службы, в реальном времени
make deploy-logs

# только прикладной сервер, последние 200 строк
docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  logs --tail=200 backend

# только ошибки
docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  logs backend | grep -i error
```

ВАЖНО. Журналы контейнеров по умолчанию не ограничены по размеру и способны заполнить диск. Настройте ограничение при вводе в эксплуатацию.

Листинг 12.3 — Ограничение размера журналов контейнеров

```
# /etc/docker/daemon.json
{
  "log-driver": "json-file",
  "log-opts": { "max-size": "50m", "max-file": "5" }
}
```

После изменения файла выполните перезапуск службы Docker и пересоздание контейнеров.

Листинг 12.4 — Применение ограничения журналов

```
sudo systemctl restart docker
make deploy-up
```

Ожидаемый результат: для каждого контейнера хранится не более пяти файлов журнала по 50 МБ.

12.3. Журналы внутри системы

Часть журналов доступна прямо в интерфейсе и не требует доступа к серверу. Их могут просматривать как специалисты ИТ, так и операторы бюро с соответствующим правом.

Таблица 12.1 — Журналы, доступные в интерфейсе

Журнал	Содержание	Где находится
Журнал запросов	Все обращения к системе: работник, действие, код ответа, длительность, время	Раздел системного управления
Журнал обращений к персональным данным	Обращения к сведениям о людях: работник, действие, раздел, адрес, время	Раздел администрирования, открывается по отдельному праву
Журнал изменений	Кто, когда и что изменил в сущностях системы	Раздел администрирования
Журнал отказов в доступе	Попытки выполнить действие без права	Раздел администрирования
Журнал входов	Входы, выходы, неудачные попытки, блокировки	Карточка работника

Журнал запросов и журнал обращений к персональным данным поддерживают фильтрацию и выгрузку в электронную таблицу.

Подробные записи журнала запросов хранятся 30 суток, затем сворачиваются в суточные итоги. Записи журнала обращений к персональным данным хранятся 36 месяцев без свёртки и вместе с журналом запросов не удаляются.

12.4. Что контролировать

Таблица 12.2 — Показатели для наблюдения

Показатель	Норма	Признак неблагополучия
Состояние служб	Все up, база healthy	Перезапуски, состояние Restarting
Свободное место на диске	Более 20 процентов	Менее 10 процентов требует немедленных действий
Сообщения уровня error в журнале	Единичные	Повторяющиеся однотипные ошибки
Отказы в доступе	Единичные	Массовые отказы указывают на неверно настроенные права
Срок действия сертификата	Более 30 суток	Менее 30 суток требует продления
Число подключений к базе	Значительно меньше 150	Приближение к пределу

13. Диагностика и типовые неисправности

13.1. Порядок диагностики

Таблица 13.1 — Команды диагностики

Что выяснить	Команда
Состояние служб	<code>docker compose -f docker-compose.base.yml -f docker-compose.prod.yml ps</code>
Причина незапуска службы	<code>docker compose -f docker-compose.base.yml -f docker-compose.prod.yml logs <служба></code>
Потребление ресурсов	<code>docker stats</code>
Свободное место	<code>df -h</code> и <code>docker system df</code>
Доступность базы	<code>docker compose -f docker-compose.base.yml -f docker-compose.prod.yml exec db pg_isready -U postgres</code>
Действующие параметры прикладного сервера	<code>docker compose -f docker-compose.base.yml -f docker-compose.prod.yml exec backend env</code>
Проверка сертификата	<code>openssl x509 -in nginx/certs/fullchain.pem -noout -dates</code>

13.2. Система недоступна, страница не открывается

Таблица 13.2 — Диагностика недоступности

Шаг	Действие	Вывод
1	Проверить состояние служб	Службы не запущены, перейти к шагу 2, иначе к шагу 3
2	<code>make deploy-up</code> , затем <code>make deploy-logs</code>	Причина незапуска будет в журнале
3	<code>curl -s http://localhost/health</code>	Ответ есть, значит неисправность в сети или правилах межсетевого экрана. Ответа нет, значит в прикладном сервере
4	<code>df -h</code>	Заполнение диска приводит к отказу базы данных

13.3. Обратный прокси не запускается

Наиболее вероятная причина: отсутствует файл сертификата. Прокси с настроенным защищённым соединением не стартует, если сертификата нет.

Листинг 13.1 — Проверка наличия сертификата

```
ls -l nginx/certs/fullchain.pem nginx/certs/privkey.pem
```

Ожидаемый результат: оба файла существуют. При их отсутствии выпустите сертификат по подразделу 7.3.

13.4. Предупреждение о недоверенном сертификате

Признак: браузер сообщает о недоверенном сертификате и не предлагает продолжить.

Отсутствие предложения продолжить нормально и вызвано требованием обязательного защищённого соединения. Причина в том, что корневой сертификат не установлен на рабочем месте, устранение по подразделу 7.4.

Листинг 13.2 — Проверка, какой сертификат отдаёт сервер

```
openssl s_client -connect <адрес>:443 -servername <адрес> \  
</dev/null 2>/dev/null \  
| openssl x509 -noout -subject -ext subjectAltName
```

Ожидаемый результат: в поле альтернативных имён присутствует адрес, по которому обращаются работники. Если обращение идёт по сетевому адресу, а сертификат выпущен только на имя, требуется перевыпуск с указанием адреса вторым аргументом.

13.5. Прикладной сервер не запускается

Таблица 13.3 — Сообщения об ошибках при запуске

Сообщение в журнале	Причина	Устранение
Ошибка обязательного параметра	Не задан или недопустим один из трёх обязательных параметров	Проверить файл параметров, подраздел 6.2
Ошибка подключения к базе	База не готова либо неверны учётные данные	Проверить доступность базы, подраздел 13.1
Ошибка приведения схемы	Несовместимое изменение схемы	Восстановить базу из копии, сделанной до обновления
Требуется ключ шифрования	Включено требование шифрования, ключ отсутствует	Задать ключ либо отключить требование

13.6. Изменение параметра не подействовало

Наиболее частая причина: параметр не входит в набор, передаваемый через файл параметров, см. подраздел 6.1.

Листинг 13.3 — Проверка действующих параметров

```
docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  exec backend env | grep RESET_TIMEZONE
```

Ожидаемый результат: выведено заданное значение. Если параметр отсутствует в выводе, добавьте его в раздел `environment` файла описания рабочего сервера.

13.7. Заканчивается место на диске

Таблица 13.4 — Освобождение дискового пространства

Потребитель	Как проверить	Как освободить
Журналы контейнеров	<code>docker system df</code>	Настроить ограничение, подраздел 12.2
Неиспользуемые образы	<code>docker system df</code>	<code>docker system prune -a --filter "until=168h"</code>
Журнал запросов в базе	Размер таблиц журнала	Уменьшить <code>REQUEST_LOG_DETAIL_DAYS</code>
Загруженные файлы	<code>docker system df -v</code>	Перенести том на отдельный раздел

ВАЖНО. Не уменьшайте срок хранения журнала обращений к персональным данным без правовых оснований. Он определяется требованиями к обработке персональных данных.

13.8. Система работает медленно

Таблица 13.5 — Диагностика замедления

Проверка	Толкование
<code>docker stats</code>	Постоянная загрузка процессора базой близко к пределу означает нехватку ресурсов либо тяжёлые отчёты
Журнал запросов, сортировка по длительности	Выявляет конкретные медленные операции
Число подключений к базе	Приближение к 150 указывает на исчерпание предела
Свободная память	Нехватка приводит к вытеснению кэша базы и резкому замедлению

13.9. Работник не видит раздел или получает отказ

1. Открыть журнал отказов в доступе: там зафиксировано, какое именно право не было предоставлено.
2. Проверить назначенные работнику роль и группы прав.

3. Учесть задержку до 30 секунд после изменения прав.
4. Убедиться, что учётная запись не заблокирована.

13.10. Не приходят обновления без перезагрузки страницы

Признак: данные на открытой странице обновляются только после ручного обновления.

Проверить, что обратный прокси не буферизует поток событий. В поставляемой конфигурации для адреса потока событий буферизация отключена и увеличен предельный срок ожидания. Если конфигурация изменялась, восстановите эти параметры.

Принудительное переподключение каждые 10 минут является штатным поведением.

Приложение А. Порты и сетевые доступы

Таблица А.1 — Сетевые доступы

Порт	Протокол	Откуда доступен	Назначение
443	TCP	Сеть организации	Работа с системой
80	TCP	Сеть организации	Перенаправление, проверка при выпуске сертификата, проверка состояния
22	TCP	Адреса подразделения ИТ	Управление сервером
8080	TCP	Только внутренняя сеть контейнеров	Прикладной сервер
80	TCP	Только внутренняя сеть контейнеров	Веб-приложение
5432	TCP	Только внутренняя сеть контейнеров	База данных

Исходящие соединения требуются только при выпуске сертификата общедоступным центром и при отправке сообщений о сбоях во внешний сервис. Без них система работает полностью автономно.

Приложение Б. Перечень параметров конфигурации

Таблица Б.1 — Подключение и запуск

Параметр	По умолчанию	Назначение
DATABASE_URL	нет, обязателен	Строка подключения к базе данных
DB_NAME	auto_registry	Имя базы данных

Продолжение таблицы Б.1

Параметр	По умолчанию	Назначение
DB_USER	postgres	Пользователь базы данных
DB_PASSWORD	нет	Пароль пользователя базы данных
BIND_HOST	0.0.0.0	Адрес приёма соединений
BIND_PORT	8090	Порт прикладного сервера, в поставке задан 8080
LOG_LEVEL	info	Подробность журнала
SWAGGER_ENABLED	false	Публикация описания программного интерфейса

Таблица Б.2 — Маркеры доступа и вход

Параметр	По умолчанию	Назначение
JWT_SECRET	нет, обязателен	Ключ подписи маркера доступа
JWT_REFRESH_SECRET	нет, обязателен	Ключ подписи маркера продления
JWT_ACCESS_TTL	15m	Срок жизни маркера доступа
JWT_REFRESH_TTL	168h	Срок жизни маркера продления
COOKIE_SECURE	true	Передача маркера продления только по защищённому соединению
LOGIN_RATE_LIMIT_MAX	10	Попыток входа с одного адреса за интервал
LOGIN_RATE_LIMIT_WINDOW_SEC	300	Интервал подсчёта попыток

Таблица Б.3 — Защита персональных данных

Параметр	По умолчанию	Назначение
DATA_ENCRYPTION_KEY	пусто	Ключ шифрования, 64 шестнадцатеричных символа
REQUIRE_ENCRYPTION	false	Запрет запуска без ключа шифрования
PD_AUDIT_RETENTION_MONTHS	36	Срок хранения журнала обращений к персональным данным

Таблица Б.4 — Ограничения и загрузка файлов

Параметр	По умолчанию	Назначение
RATE_LIMIT_PER_MINUTE	200	Запросов в минуту на работника
RATE_LIMIT_WINDOW_SEC	60	Интервал подсчёта

Продолжение таблицы Б.4

Параметр	По умолчанию	Назначение
PAGINATION_MAX_LIMIT	100	Наибольший размер страницы выдачи
UPLOAD_PATH	./uploads	Каталог хранения файлов, в поставке /app/uploads
UPLOAD_MAX_FILE_SIZE	10485760	Наибольший размер файла в байтах
UPLOAD_ALLOWED_IMAGE_TYPES	image/jpeg, image/png, image/webp	Разрешённые типы изображений
UPLOAD_ALLOWED_DOC_TYPES	application/pdf и формат документов	Разрешённые типы документов

Таблица Б.5 — Журналы и обслуживание

Параметр	По умолчанию	Назначение
LOG_FILE_PATH	пусто	Файл журнала, пустое значение означает вывод только в поток
LOG_MAX_SIZE_MB	100	Размер файла до смены
LOG_MAX_AGE_DAYS	30	Срок хранения файлов журнала
LOG_MAX_BACKUPS	14	Число хранимых файлов
LOG_COMPRESS	true	Сжатие при смене
REQUEST_LOG_DETAIL_DAYS	30	Глубина подробных записей журнала запросов
REQUEST_LOG_PARTITION_PRECREATE_DAYS	7	На сколько суток вперёд создаются разделы
ANALYTICS_CACHE_REFRESH_SEC	60	Периодичность пересчёта показателей

Таблица Б.6 — Прочие параметры

Параметр	По умолчанию	Назначение
CORS_ALLOWED_ORIGINS	http://localhost:8081	Адреса, которым разрешены обращения к интерфейсу
RESET_TIMEZONE	Europe/Moscow	Часовой пояс суточного сброса статусов
TELEGRAM_BOT_TOKEN	пусто	Отправка сообщений о сбоях во внешний сервис
TELEGRAM_CHAT_ID	пусто	Адресат сообщений о сбоях
COMPOSE_PROJECT_NAME	имя каталога	Имя установки, определяет имена томов с данными

Приложение В. Файлы, необходимые для запуска

Перечень содержимого поставки, без которого система не соберётся и не запустится.

Таблица В.1 — Обязательные файлы и каталоги

Путь	Назначение
docker-compose.base.yml	Описание состава системы, общее для стенда и рабочего сервера
docker-compose.prod.yml	Дополнение для рабочего сервера: параметры базы, ресурсы, обратный прокси
docker-compose.staging.yml	Дополнение для проверочного стенда
Dockerfile	Сборка образа прикладного сервера
frontend/Dockerfile	Сборка образа веб-приложения
frontend/nginx.conf	Конфигурация отдачи статики внутри образа веб-приложения
nginx/nginx.conf	Конфигурация обратного прокси рабочего сервера
nginx/staging.conf	Конфигурация обратного прокси стенда
nginx/Dockerfile.staging	Сборка образа прокси для стенда: базовый образ дополняется средством создания файла паролей
Makefile	Команды управления
scripts/init-env.sh	Создание файла параметров
scripts/init-tls.sh	Выпуск и продление сертификата
go.mod, go.sum	Перечень библиотек серверной части
cmd/, internal/, docs/	Исходный код серверной части и описание программного интерфейса
frontend/package.json, frontend/package-lock.json	Перечень библиотек интерфейса
frontend/src/, frontend/index.html, frontend/vite.config.js	Исходный код интерфейса

Создаются на сервере при установке и в поставку не входят:

Таблица В.2 — Создаваемое при установке

Путь	Чем создаётся
.env	make init-production
nginx/certs/	scripts/init-tls.sh
nginx/acme-webroot/	scripts/init-tls.sh при выпуске через общедоступный центр

Приложение Г. Проверочный лист ввода в эксплуатацию

Таблица Г.1 — Проверочный лист

№	Проверка	Отметка
1	Операционная система обновлена, часовой пояс задан	
2	Docker и Docker Compose установлены, версии соответствуют требованиям	
3	Дисковый массив собран в зеркало	
4	Файл параметров создан, секреты переданы в хранилище организации	
5	Ключ шифрования сохранён отдельно от сервера и от резервных копий	
6	Имя базы данных задано до первого запуска	
7	Сертификат выпущен, срок действия проверен	
8	Корневой сертификат установлен на рабочие места, если применялся собственный центр	
9	Система запущена, все службы работают	
10	Проверка состояния отвечает	
11	Обращение по незащищённому протоколу перенаправляется	
12	Учётная запись администратора создана, пароль отличается от исходного	
13	Вход выполняется, разделы открываются	
14	Правила межсетевого экрана настроены по приложению А	
15	Ограничение размера журналов контейнеров настроено	
16	Резервное копирование настроено, восстановление проверено	
17	Мониторинг подключён	
18	Демонстрационные данные на рабочем сервере отсутствуют	
19	Публикация описания программного интерфейса отключена	
20	Требование шифрования персональных данных включено	